МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ВТ

ОТЧЕТ
по лабораторной работе № 3
по дисциплине «Распределённые системы»
Тема: Обеспечение безопасности при межсервисном взаимодействии в распределенных системах на примере микросервисной архитектуры

Студенты гр. 2307	_____	Почекунин А. П.
	_____	Буянов. М.Г.
	_____	Подберёзский А.Д.
Преподаватель	_____	Соколова Е.А.

Санкт-Петербург
2026

Цель работы: Научиться реализовывать базовые механизмы безопасности при взаимодействии двух микросервисов: аутентификацию с использованием токенов и шифрование данных при передаче.

Задание:

В ходе выполнения лабораторной работы необходимо разработать и реализовать модель безопасного взаимодействия между двумя микросервисами. Для этого необходимо разработать минимум 2 независимых микросервиса:

1. Сервис аутентификации - микросервис, отвечающий за выдачу валидных токенов доступа. Другие сервисы системы должны обращаться к нему для подтверждения подлинности запросов.
2. Сервис данных - микросервис, предоставляющий доступ к защищённым данным. Он должен принимать запросы только от авторизованных пользователей или сервисов, проверяя валидность предоставленного токена.

Также, необходимо выполнить реализацию механизмов аутентификации, обеспечить безопасный обмен данными между микросервисами, а также организовать логирование всех входящих запросов и попыток доступа к данным для последующего аудита.

Сервис аутентификации:

Для реализации механизма безопасности в распределённой системе был разработан микросервис аутентификации на языке Python с использованием веб-фреймворка FastAPI.

Принцип работы сервиса заключается в следующем: сервис аутентификации запускается как отдельное приложение и предоставляет конечную точку (эндпоинт) для входа в систему. Пользователь или другой микросервис отправляет HTTP-запрос с учётными данными (имя пользователя и пароль) на эндпоинт /login. Сервис аутентификации принимает запрос, проверяет наличие учётной записи во встроенной базе данных пользователей, и в случае успеха генерирует JWT-токен (JSON Web Token) с заданным временем жизни. Сформированный токен возвращается клиенту в ответе на запрос.

В ходе реализации использовался веб-фреймворк FastAPI, библиотека python-jose для работы с JWT-токенами и встроенный модуль logging для записи событий. Взаимодействие осуществляется по протоколу HTTP, при этом сервис аутентификации выступает центральным элементом системы, отвечающим за проверку подлинности пользователей и выдачу токенов доступа.

Важной особенностью разработанного сервиса является логирование всех попыток входа в систему: в файл auth.log записываются временные метки, имена пользователей и результаты проверки учётных данных. Это позволяет в дальнейшем анализировать попытки несанкционированного доступа и обеспечивает возможность аудита безопасности.

Сервис аутентификации не хранит никаких других данных, кроме учётных записей пользователей, и не взаимодействует напрямую с другими микросервисами — его единственная функция заключается в генерации токенов, которые затем используются для доступа к защищённым ресурсам системы.

Также, для наглядной демонстрации работы и взаимодействия сервисов было реализовано простое веб-приложение.

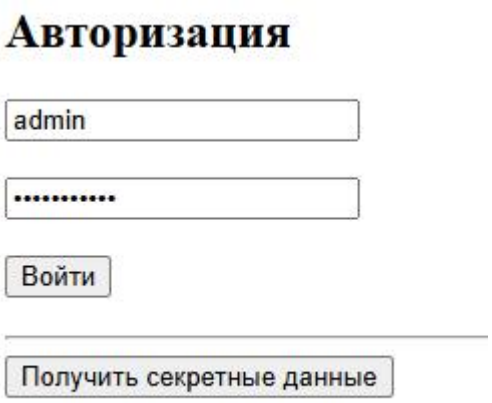

Рис.1. Внешний вид веб-приложения для демонстрации работы сервисов

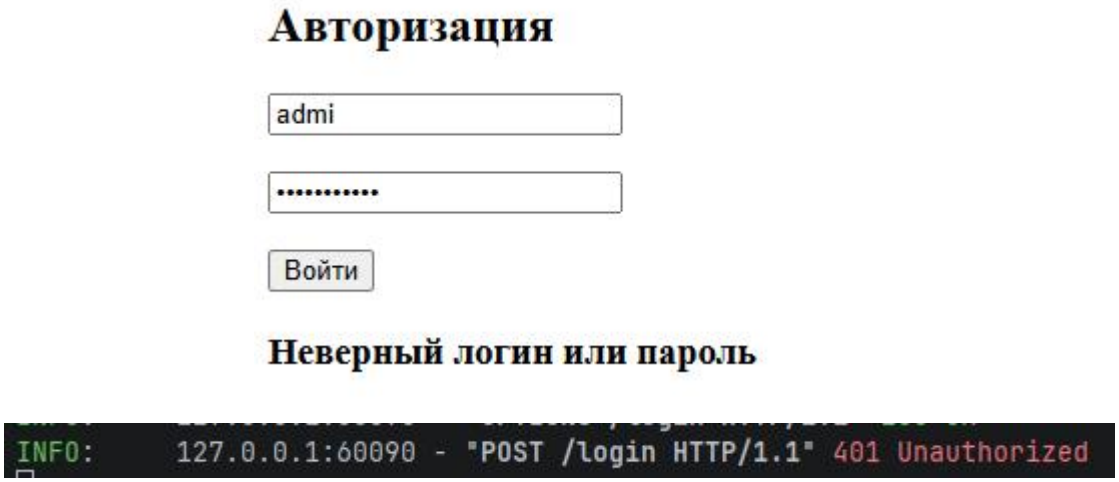

Рис.2. Результат ввода неверного логина и/или пароля

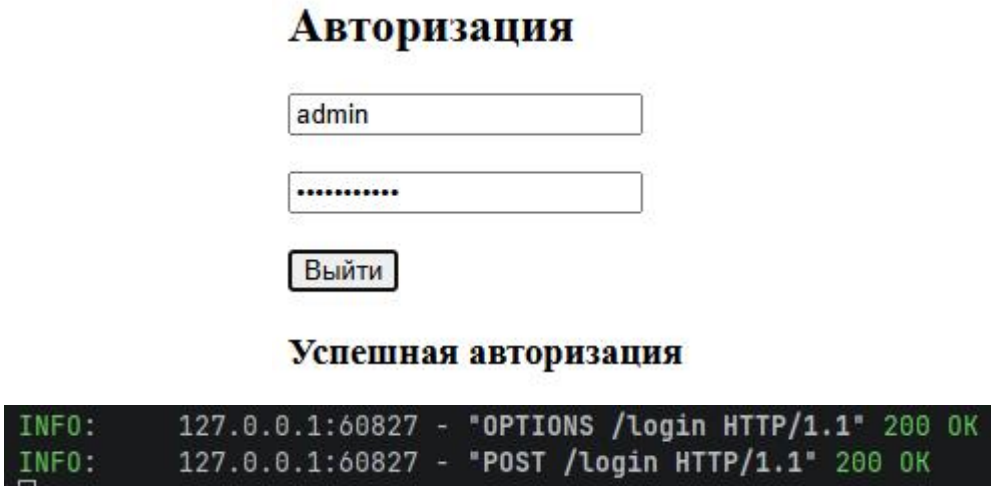

Рис.3. Результат ввода верных логина и пароля

Сервис данных:

Для реализации защищённого доступа к информации в распределённой системе был разработан микросервис данных на языке Python с использованием веб-фреймворка FastAPI.

Принцип работы сервиса заключается в следующем: сервис данных запускается как отдельное приложение и предоставляет конечную точку (эндпоинт) для получения защищённой информации. Клиент (пользователь или другой микросервис) отправляет HTTP-запрос на эндпоинт /data, обязательно указывая в заголовке Authorization JWT-токен, полученный ранее от сервиса аутентификации. Сервис данных принимает запрос, извлекает токен из заголовка и проверяет его валидность: расшифровывает токен с использованием общего секретного ключа и проверяет срок его действия. В случае успешной проверки сервис идентифицирует пользователя и возвращает запрашиваемые данные. Если токен отсутствует, недействителен или истёк, сервис отклоняет запрос и возвращает ошибку доступа.

В ходе реализации использовался веб-фреймворк FastAPI, библиотека python-jose для проверки JWT-токенов, а также библиотека cryptography для шифрования возвращаемых данных с использованием симметричного алгоритма Fernet. Взаимодействие осуществляется по протоколу HTTP, при этом сервис данных полагается на сервис аутентификации для проверки подлинности пользователей и самостоятельно не хранит никаких учётных записей.

Важной особенностью разработанного сервиса является двухуровневая защита данных. Во-первых, доступ к эндпоинту возможен только при наличии валидного токена, что обеспечивает аутентификацию и авторизацию. Во-вторых, даже после успешной аутентификации возвращаемые данные дополнительно шифруются с использованием уникального ключа, что гарантирует их конфиденциальность при передаче. Ключ шифрования также возвращается клиенту для возможности расшифровки полученной информации.

Сервис данных также реализует логирование всех попыток доступа: в файл data.log записываются временные метки, результаты проверки токенов и

информация об успешных или неуспешных попытках получения данных. Это позволяет отслеживать подозрительную активность и обеспечивает возможность последующего анализа безопасности системы.

Дополнительно в сервисе реализован отдельный эндпоинт /log-unauthorized, который предназначен для логирования попыток несанкционированного доступа, например, со стороны веб-интерфейса, когда клиент даже не пытается предоставить токен.

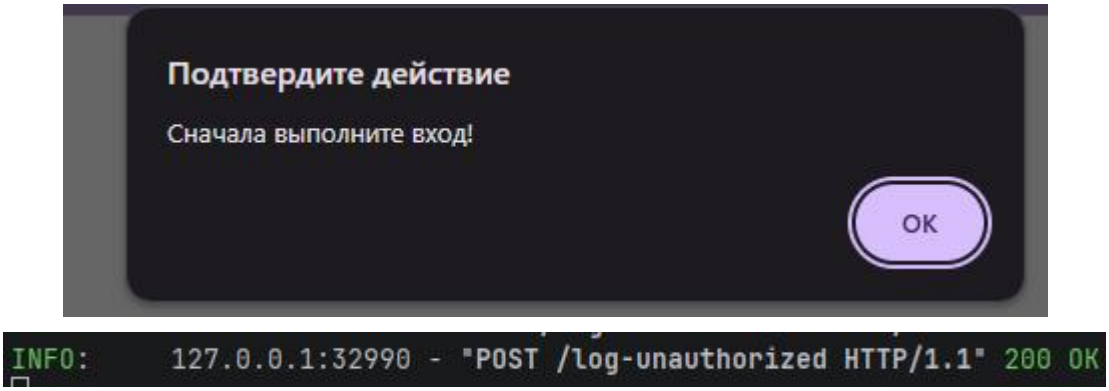

Рис. 4. Результат получения данных неавторизованным пользователем

Рис. 5. Результат получения данных пользователем admin

Вывод:

В ходе выполнения лабораторной работы были изучены и реализованы базовые механизмы обеспечения безопасности при межсервисном взаимодействии в распределённых системах на примере микросервисной архитектуры. Разработанные сервисы аутентификации и данных продемонстрировали эффективность применения JWT-токенов для разграничения доступа и шифрования передаваемой информации.

Реализованный подход показал, что централизованная аутентификация через выделенный сервис упрощает управление доступом и повышает безопасность системы, однако создаёт единую точку отказа, требующую дополнительных мер по обеспечению отказоустойчивости. Логирование всех запросов и попыток доступа обеспечивает возможность аудита и анализа инцидентов, что является необходимым элементом защиты распределённых приложений.

Таким образом, предложенная схема взаимодействия микросервисов с использованием токенов и шифрования позволяет эффективно контролировать доступ к данным и обеспечивать их конфиденциальность, однако выбор конкретных механизмов безопасности должен определяться требованиями к надёжности, масштабируемости и уровню угроз для конкретной системы.

Приложение 1. Код сервиса авторизации

```
from fastapi import FastAPI, HTTPException
from jose import jwt
from datetime import datetime, timedelta
import logging
from pydantic import BaseModel
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

SECRET_KEY = "SUPER_SECRET_KEY"
ALGORITHM = "HS256"
TOKEN_EXPIRE_MINUTES = 30

users_db = {
    "admin": "password123"
}

class LoginRequest(BaseModel):
    username: str
    password: str

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
logging.basicConfig(
    filename="auth.log",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)

def create_token(username: str):
    payload = {
        "sub": username,
        "exp": datetime.utcnow() + timedelta(minutes=TOKEN_EXPIRE_MINUTES)
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

@app.post("/login")
def login(data: LoginRequest):
    username = data.username
    password = data.password
    logging.info(f"Login attempt: {username}")
    if users_db.get(username) != password:
        logging.warning(f"FAILED login: {username}")
        raise HTTPException(status_code=401, detail="Invalid credentials")
    token = create_token(username)
    logging.info(f"SUCCESS login: {username}")
    return {"access_token": token}
```

Приложение 2. Код сервиса данных

```
from fastapi import FastAPI, Header, HTTPException
from jose import jwt, JWTError
from cryptography.fernet import Fernet
import logging
from fastapi.middleware.cors import CORSMiddleware
```

```
app = FastAPI()

SECRET_KEY = "SUPER_SECRET_KEY"
ALGORITHM = "HS256"
FERNET_KEY = Fernet.generate_key()
cipher = Fernet(FERNET_KEY)

logging.basicConfig(
    filename="data.log",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

def verify_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload["sub"]
    except JWTError:
        return None

@app.post("/log-unauthorized")
```

```
def log_unauthorized():
    logging.warning("Unauthorized access attempt from web page (no token)")
    return {"status": "logged"}

@app.get("/data")
def get_secure_data(authorization: str = Header(None, alias="Authorization")):

    logging.info("Access attempt to /data")

    if not authorization:
        logging.warning("No token provided")
        raise HTTPException(status_code=401, detail="Token required")

    token = authorization.replace("Bearer ", "")
    user = verify_token(token)

    if not user:
        logging.warning("Invalid token")
        raise HTTPException(status_code=401, detail="Invalid token")

    logging.info(f"Access granted for {user}")

    secret_data = f"Very secret data for {user}"
    encrypted = cipher.encrypt(secret_data.encode())

    return {
        "massage": f"Hello, {user}! Here is your encrypted data.",
        "secret_data": secret_data,
        "encrypted_data": encrypted.decode(),
```

```
        "encryption_key": FERNET_KEY.decode()
    }
```

Приложение 3. Код страницы авторизации

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Secure Microservices Demo</title>
</head>
<body>
<h2>Авторизация</h2>
<input id="username" placeholder="Username" value="admin">
<br><br>
<input id="password" type="password" placeholder="Password"
value="password123">
<br><br>
<button onclick="login()">Войти</button>
<h3 id="loginStatus"></h3>
<hr>
<button onclick="getData()">Получить секретные данные</button>
<h3 id="result"></h3>
<script>
let token = null;
let loggedIn = false;
async function login() {
    if (loggedIn) {
        token = null;
        loggedIn = false;
        document.getElementById("loginStatus").innerText =
            "Вы вышли из аккаунта";
```

```
        document.querySelector("button").innerText = "Войти";
        document.getElementById("result").innerText = "";

        return;
    }
    const username = document.getElementById("username").value;
    const password = document.getElementById("password").value;
    const response = await fetch("http://localhost:8000/login", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            username: username,
            password: password
        })
    });
    if (!response.ok) {
        document.getElementById("loginStatus").innerText =
            "Неверный логин или пароль";
        return;
    }
    const data = await response.json();
    token = data.access_token;
    loggedIn = true;
    document.getElementById("loginStatus").innerText =
        "Успешная авторизация";
    document.querySelector("button").innerText = "Выйти";
}
async function getData() {
```

```
    if (!token) {
        await fetch("http://localhost:8001/log-unauthorized", {
            method: "POST"
        });
        alert("Сначала выполните вход!");
        return;
    }
    const response = await fetch("http://localhost:8001/data", {
        method: "GET",
        headers: {
            "Authorization": "Bearer " + token
        }
    });
    const data = await response.json();
    document.getElementById("result").innerText =
        data.massage +
        "\n\nEncrypted data:\n" + data.encrypted_data +
        "\n\nEncryption key:\n" + data.encryption_key +
        "\n\nSecret data:\n" + data.secret_data;
}
</script>
</body>
</html>
```